

EFREI Paris

School of Engineering and Computer Science

Academic Year 2024–2025

Final Exam

Big Data Engineering

XICS404

Practical project:

*Designing an end-to-end Data Engineering
platform for IoT sensor data*

Date: Tuesday 19th

Time slot: 8:00 – 11:00 AM

Duration: 3 hours

Format: Individual practical project

Total grade: /100 points

Allowed tools: Documentation, generative AI, IDE

Preamble and exam rules

⚠ Read carefully before starting

- The exam runs for **3 hours (8:00 – 11:00 AM)** with no extended break.
- The use of **generative AI tools** (ChatGPT, Claude, Copilot, etc.) is **permitted**. The exam has been calibrated accordingly: code produced by an AI alone is not enough to earn a strong grade.
- The assessment focuses on your ability to **design, integrate, debug and justify** a complete data pipeline. The sections requiring technical justification carry significant weight.
- **Collaboration between students is prohibited**. Any communication (chat, e-mail, screen sharing) between candidates leads to exclusion.
- Code and documentation must be written in **English or French**, consistently across the whole deliverable.
- Git commits dated **after 11:00 AM** will not be considered.

i Methodological advice

Do not try to code everything in detail in the first minutes. Start by: (1) reading the whole exam paper, (2) sketching the architecture, (3) launching the cluster, (4) implementing a minimal working version, and only then (5) enriching each layer progressively. A simple end-to-end pipeline that works is worth far more than a sophisticated but isolated single layer.

Contents

Preamble and rules	1
1 Project context	3
1.1 Scenario	3
1.2 Your mission	3
1.3 Functional scope	3
2 Learning objectives assessed	4
3 Prerequisites and technical environment	4
3.1 Mandatory tools and versions	4
3.2 Kafka cluster	4
3.3 Data conventions	5
4 Target architecture	5
4.1 Overview	6
4.2 Kafka topic	6
4.3 Message format	6
4.4 Data lake structure	6
5 Work required	7
5.1 Part 1 — Infrastructure and Kafka topic [10 pts] (20 min) . . .	7
5.2 Part 2 — Python producer [15 pts] (30 min)	7
5.3 Part 3 — Spark processing pipeline [25 pts] (60 min)	8
5.4 Part 4 — Analytical queries with Spark SQL [15 pts] (30 min) . . .	9
5.5 Part 5 — REST API [15 pts] (30 min)	10
5.6 Part 6 — Documentation and justification [20 pts] (30 min) . . .	10
6 Expected deliverables	11
6.1 Repository structure	11
6.2 Submission rules	12
7 Technical constraints	12
8 Detailed grading grid	12
8.1 Overall scoring	13
8.2 Transverse criteria (bonus / penalty)	13
9 Indicative time allocation	13
10 Reflection questions (up to +5 bonus points)	13
Appendix A: Useful commands	15
Appendix B: Submission checklist	15

1. Project context

1.1 Scenario

You have just joined the Data Platform team of **AeroSense**, a company specialising in industrial environmental monitoring. AeroSense deploys IoT sensor networks at its customer sites (factories, warehouses, data centres) to continuously collect three physical measurements: **temperature**, **relative humidity**, and **atmospheric pressure**.

Each sensor emits readings at a high frequency. Across a dozen sites, this represents several thousand events per second. The business stakeholders (*Operations* and *Predictive Maintenance* teams) require:

- reliable, fault-tolerant **real-time ingestion**,
- **historical storage** enabling analysis over days or months,
- **programmatic access** to statistics and latest readings,
- **anomaly detection** to trigger operational alerts.

1.2 Your mission

Your mission is to design and implement an end-to-end **Data Engineering platform** covering the full data lifecycle:

Generation → Ingestion (Kafka) → Processing (Spark)
→ Storage (Data Lake) → Exposure (REST API) → Consumption

The platform must be runnable by your reviewer on their own machine in a few commands, starting from your repository.

1.3 Functional scope

🎯 Business use case

1. Sensors emit their readings to the central system.
2. Each reading is ingested into Kafka, partitioned by sensor type.
3. A Spark Structured Streaming job:
 - parses the JSON payloads,
 - cleans outlier values,
 - detects anomalies (temperature and humidity beyond thresholds),
 - computes rolling 5-minute averages,
 - writes the results as partitioned Parquet to a local data lake.
4. A REST API exposes these results to consuming applications.
5. A final analytical pass leverages the data lake to produce a summary report.

2. Learning objectives assessed

This project assesses your mastery of the following competencies, covered across the five course sessions:

navy!85		
S1	Distributed architecture, multi-broker Kafka cluster deployment, CAP principles, replication, ISR.	10%
S2	Python producers and consumers, fine-grained offset management, delivery semantics, consumer groups.	15%
S3	Spark Structured Streaming, JSON parsing, watermarking, time windows, checkpointing, Parquet sinks.	25%
S4	Three-zone architecture (raw / curated / consumption), partitioning, Spark SQL queries, partition pruning.	20%
S5	REST API design, input validation, HTTP status codes, JSON serialisation, integration with data lake and Kafka.	15%
Transverse	Documentation, overall architecture, justification of design choices, code quality, reproducible execution.	15%

3. Prerequisites and technical environment

3.1 Mandatory tools and versions

navy!85		
Docker / Docker Compose	20.10+ / v2.0+	Kafka cluster
Python	3.9+	Producer, consumer, API
Apache Kafka	7.5 (Confluent)	Message bus
PySpark	3.5.x	Streaming and batch processing
Flask	3.0+	REST API
kafka-python-ng	latest	Python Kafka client

3.2 Kafka cluster

You will **reuse the 3-broker Kafka cluster in KRaft mode** set up during Session 1. The configuration must include:

- 3 Kafka brokers (`kafka1`, `kafka2`, `kafka3`),
- a `replication-factor` of 3 and `min.insync.replicas` of 2,
- the Kafka UI accessible at <http://localhost:8080>.

3.3 Data conventions

Reuse of data conventions from previous sessions

To keep the exam focused on architecture and integration, the **data conventions** used across the platform are the same as those you have manipulated in Sessions 2, 3 and 4. There is no new schema to learn:

- The **message schema** is the one introduced in Session 2: a JSON payload with fields `sensor`, `value`, `unit`, `timestamp`, `source`, `anomaly`.
- The **Kafka topic** is `sensor-events`.
- The **three sensor types** are `temperature`, `humidity` and `pressure`, with units `C`, `%` and `hPa` and the realistic ranges you already know.
- The **Hive-style partitioning conventions** for the data lake follow the layout you used in Session 4.

Code reuse is NOT allowed

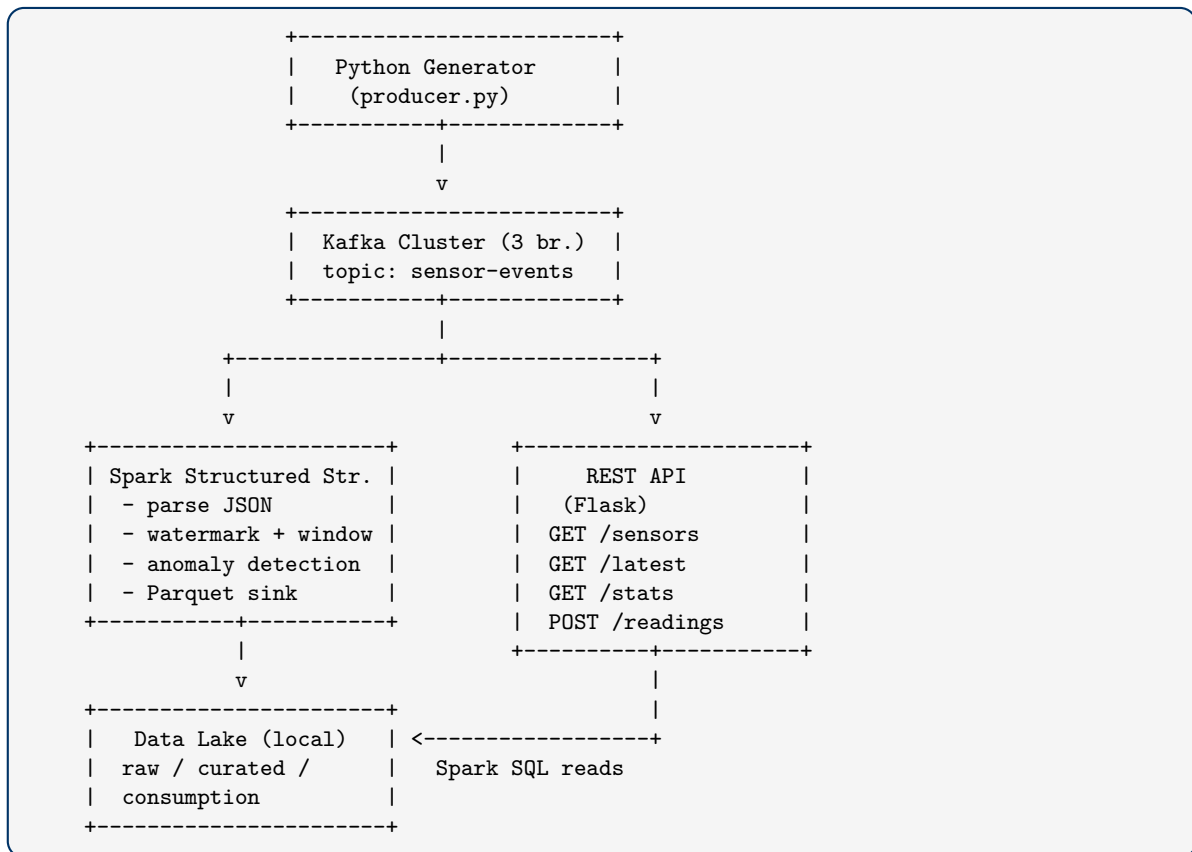
Although the *data* and the *conventions* come from previous sessions, you must **rewrite all the code from scratch** for this exam. Specifically:

- Do **not** copy or paste any code from your previous lab solutions (`producer.py`, `consumer.py`, `etl_pipeline.py`, `datalake_pipeline.py`, etc.).
- Do **not** copy the code listings from the lab handouts verbatim.
- Every Python file, every Spark job and every Flask endpoint in your deliverable must be written during the exam.

The lab handouts can be used as a *reference* for syntax and concepts, but the implementation must be your own work. Submissions that show clear evidence of copy-paste from previous labs will incur the plagiarism penalty (see Section 8).

4. Target architecture

4.1 Overview



4.2 Kafka topic

You will create a topic named **sensor-events** with:

- 3 partitions,
- a replication factor of 3,
- key-based partitioning on the **sensor_type**.

4.3 Message format

Every message exchanged on Kafka must follow this JSON schema:

```

1 {
2   "sensor":    "temperature", # or "humidity", "pressure"
3   "value":     27.45,         # float
4   "unit":      "C",           # str
5   "timestamp": 1737543600000, # epoch ms
6   "source":    "site-A-rack-12",
7   "anomaly":   false          # boolean, computed by the producer
8 }

```

Listing 1: Sensor event schema

4.4 Data lake structure

```
1 /tmp/datalake/  
2 |-- raw/  
3 |   '-- source=kafka/topic=sensor-events/  
4 |   '-- year=YYYY/month=MM/day=DD/hour=HH/  
5 |-- curated/  
6 |   '-- domain=iot/  
7 |   '-- sensor_type=.../year=YYYY/month=MM/day=DD/  
8 '-- consumption/  
9   '-- use_case=sensor_averages/  
10   '-- sensor_type=.../year=YYYY/month=MM/
```

Listing 2: Mandatory data lake layout

5. Work required

The project is split into **six parts**. An indicative time budget is given for each.

5.1 Part 1 — Infrastructure and Kafka topic [10 pts] (20 min)

Objective

Bring up the Kafka cluster, create the **sensor-events** topic, and demonstrate fault tolerance.

Tasks

1. Reuse (or recreate) the `docker-compose.yml` from Session 1 and start the cluster.
2. Create the **sensor-events** topic (3 partitions, replication factor 3).
3. Inspect leader distribution across brokers using `kafka-topics -describe`.
4. Run a fault tolerance test: stop one broker, observe leader re-election, and capture a text trace (`describe` before / after) in `docs/fault_tolerance.md`.

Grading criteria

- The cluster starts without errors (4 points).
- The topic is correctly configured (2 points).
- The fault tolerance test is documented with before/after evidence (4 points).

5.2 Part 2 — Python producer [15 pts] (30 min)

Objective

Implement, **from scratch**, a Python producer (`producer.py`) that emits realistic sensor events to the **sensor-events** topic, following the data conventions of Sessions 2 and 3.

Tasks

1. The producer must generate readings for the three known sensor types: **temperature**, **humidity**, **pressure**.
2. Values must be drawn randomly within the realistic ranges already used in the labs:
 - temperature: 15 – 45 °C

- humidity: 30 – 95 %
 - pressure: 980 – 1040 hPa
3. Emit messages strictly following the agreed JSON schema (`sensor`, `value`, `unit`, `timestamp`, `source`, `anomaly`).
 4. The producer must be configured with:
 - `acks='all'`
 - `retries=5`
 - `max_in_flight_requests_per_connection=1`
 - consistent `linger_ms` and `batch_size` values for throughput
 5. The message key must be the sensor type, to guarantee ordering per type.
 6. The producer must accept a `-count N` argument (number of events to produce), a `-rate` argument (events per second) and a `-source` argument (site identifier such as `site-A-rack-12`).
 7. **At least 10% of messages** must be anomalies (out-of-threshold values), so the detection chain implemented in Part 3 can be tested.

Grading criteria

- Correct reliability configuration (4 points).
- Realistic, parameterised generation (4 points).
- Key-based partitioning demonstrated (3 points).
- Clean error handling and flush on exit (4 points).

5.3 Part 3 — Spark processing pipeline

[25 pts]**(60 min)**

Objective

Build, **from scratch**, a Spark Structured Streaming job (`spark_pipeline.py`) that consumes `sensor-events` from Kafka, processes the events, and feeds the three-zone data lake (raw, curated, consumption).

Tasks

1. **Kafka read:** subscribe to the `sensor-events` topic in streaming mode.
2. **Parsing:** use `from_json` with an explicit schema; reject invalid records.
3. **Validation:** filter outlier values (values outside physical plausibility ranges).
4. **Anomaly detection:** add an `is_anomaly` column with the following rule (computed independently of the producer's self-declared flag):
 - temperature > 35°C OR
 - humidity > 90% OR
 - pressure < 990 hPa OR > 1030 hPa.
5. **Windowed aggregation:** per 5-minute window and per `sensor_type`, compute:
 - the mean, min, max,

- the number of observations,
 - the number of detected anomalies.
6. **Watermark:** apply a 2-minute watermark on `event_time`.
 7. **Triple write** to the data lake (the layout is the one introduced in Session 4):
 - **Raw zone:** raw JSON, partitioned by *ingestion year/month/day/hour*.
 - **Curated zone:** Parquet (Snappy compression), partitioned by `sensor_type/year/month/day` based on *event time*.
 - **Consumption zone:** Parquet containing the windowed aggregates, partitioned by `sensor_type`.
 8. **Checkpointing:** configure a separate checkpoint directory for each sink.

Grading criteria

- Kafka read and parsing with explicit schema (4 points).
- Correct validation and anomaly detection (4 points).
- Windowed aggregation + watermark working (6 points).
- All three data lake zones correctly populated (8 points).
- Suitable checkpointing and `outputMode` choices, justified in the docs (3 points).

5.4 Part 4 — Analytical queries with Spark SQL [15 pts] (30 min)

Objective

Query the data lake with Spark SQL and demonstrate the impact of partition pruning.

Tasks

Create a script `analytics.py` that executes (at minimum) the following queries:

1. Top 5 hours with the highest number of anomalies, all sensors combined.
2. For each sensor type: global mean, min, max, standard deviation, anomaly rate as a percentage.
3. Daily evolution of the mean and anomaly count for the `temperature` sensor.
4. **Partition pruning demonstration:** run the same count query with and without filters on partition columns, measure both execution times, and compute the speedup factor.

Results must be:

- printed to standard output,
- written as CSV files in `outputs/analytics/`,
- commented in `docs/analytics.md`.

Grading criteria

- All four queries run and produce coherent results (8 points).
- Quantified partition pruning demonstration (4 points).
- Explanatory comments in the documentation (3 points).

5.5 Part 5 — REST API**[15 pts]****(30 min)****Objective**

Expose the platform through a Flask API that follows REST principles.

Endpoints to implement

navy!85			
GET	/api/v1/health	Health check	200
GET	/api/v1/sensors	List of sensor types	200
GET	/api/v1/sensors/<type>/latest	Latest Kafka reading	200/404
GET	/api/v1/sensors/<type>/stats?days=N	Daily stats (Parquet)	200/400/404
GET	/api/v1/anomalies?sensor=<type>&limit=N	List of recent anomalies	200/400
POST	/api/v1/readings	Publish a reading to Kafka	201/400/422

Requirements

- All responses (success and error) must be JSON with a consistent structure.
- Strict input validation: sensor type within the allowed list, `days` integer between 1 and 90, numeric values for readings.
- Correct distinction between 400 (malformed) and 422 (semantically invalid).
- Global handlers for 404, 405 and 500 returning JSON.
- Server-side error logging using `app.logger`.

Grading criteria

- All 6 endpoints work and return the correct status codes (8 points).
- Rigorous input validation (3 points).
- Consistent error handling (2 points).
- `curl` test commands provided in the documentation (2 points).

5.6 Part 6 — Documentation and justification**[20 pts]****(30 min)****Objective**

Produce technical documentation worthy of a professional deliverable.

Expected content

A README.md file at the root of the project must contain:

1. **Overview:** objectives, scope, technologies used (1 page max).
2. **Architecture:** pipeline diagram (ASCII art or image), description of each component.
3. **Instructions:** prerequisites, installation, step-by-step execution, test commands.
4. **Technical choices:** for each of the items below, explain in 3 to 5 lines *why* the choice was made and *which alternatives* were considered:
 - The partitioning strategy chosen for the curated zone.
 - The Spark Structured Streaming `outputMode`.
 - The replication factor and `min.insync.replicas` setting.
 - The use of `event_time` vs `ingestion_time` across zones.
 - The end-to-end delivery semantics (at-most-once / at-least-once / exactly-once) chosen, and their limitations.
5. **Results:** numeric excerpts from analytical queries, Kafka UI screenshots showing lag, sample API responses.
6. **Limitations and improvements:** honestly, what was not finished, and what you would do with two extra days.

Grading criteria

- Clear overview and architecture (4 points).
- Reproducible installation instructions (5 points).
- Relevant technical justifications (8 points).
- Quantified results provided (3 points).

! Important for this part

The Technical choices section is the most discriminating one. An AI can produce working Spark code; it cannot convincingly explain *your* choices in *your* context without you having understood what you wrote. Be ready to defend every choice should a follow-up oral assessment be required.

6. Expected deliverables

6.1 Repository structure

You will submit a compressed folder named `LASTNAME_FIRSTNAME_exam.zip` with the following structure:

```
1 LASTNAME_FIRSTNAME_exam/  
2 |-- README.md  
3 |-- docker-compose.yml  
4 |-- requirements.txt  
5 |-- docs/  
6 |   |-- architecture.md
```

```

7 | |-- fault_tolerance.md
8 | |-- analytics.md
9 | '-- reflection.md
10 |-- src/
11 | |-- producer.py
12 | |-- consumer.py           (optional, for testing)
13 | |-- spark_pipeline.py
14 | |-- analytics.py
15 | '-- api/
16 | |-- app.py
17 | |-- kafka_utils.py
18 | '-- lake_utils.py
19 |-- outputs/
20 | |-- analytics/           (CSV files from queries)
21 | '-- screenshots/        (Kafka UI, curl, etc.)
22 '-- tests/
23 | '-- test_curl_commands.sh (curl commands to test the API)

```

Listing 3: Deliverable layout

6.2 Submission rules

- The zipped repository must be uploaded to the e-learning platform **before 11:00 AM sharp**.
- No submissions via e-mail will be accepted.
- Late submissions incur a penalty of **5 points per 5-minute interval**.

7. Technical constraints

✓ Must be respected

- The project must be runnable by your reviewer with a simple sequence: `docker compose up -d` then running the Python scripts in the documented order.
- All Python dependencies must be listed in `requirements.txt` with pinned versions.
- No absolute user paths (`/home/yourname/...`) anywhere in the code. Use environment variables or constants at the top of files.
- The code must run on a standard Linux/macOS machine or WSL2 with no modification.

✗ Prohibitions

- Using paid cloud services (Confluent Cloud, Databricks, managed AWS). Everything must run locally.
- Third-party libraries not listed in `requirements.txt`.
- Massive copy-paste from course materials without adaptation: similarity will be detected and penalised.
- Disabling Kafka reliability checks (`acks=0, replication-factor 1`) to gain performance.

8. Detailed grading grid

8.1 Overall scoring

navy!85		
1	Kafka infrastructure and fault tolerance	10
2	Parameterised and reliable Python producer	15
3	Complete Spark Structured Streaming pipeline	25
4	Analytical queries and partition pruning	15
5	Flask REST API	15
6	Documentation and technical justification	20
Total		100

8.2 Transverse criteria (bonus / penalty)

navy!85		
Reproducibility	The project starts without modification at the reviewer's side	+3
Code quality	Naming, modularity, useful comments	+2
Over-engineering	Unjustified complexity masking design errors	-2
Non-functional	A deliverable that does not run	-10
Plagiarism / raw AI	Code visibly not understood by the student	up to -100

9. Indicative time allocation

navy!85			
8:00 – 8:15	Reading the exam, planning	15 min	Do not skip this step!
8:15 – 8:35	Part 1: cluster + topic	20 min	Start early, image pulls take time.
8:35 – 9:05	Part 2: producer	30 min	Run it and verify via Kafka UI.
9:05 – 10:05	Part 3: Spark pipeline	60 min	The longest part – tackle it with energy.
10:05 – 10:35	Part 4: analytics	30 min	The data lake must already contain data.
10:35 – 10:50	Part 5: REST API	15–30 min	If late, deliver at least 3 endpoints.
10:50 – 11:00	Documentation + zip	10 min	Do not neglect the docs!

Fallback strategy

If by 10:00 AM your Spark pipeline is still not running, do not insist. Switch to the API and read the *already partially written* data lake if at least a few Parquet files exist. A partial but coherent platform beats a perfect but isolated pipeline.

10. Reflection questions (up to +5 bonus points)

To be answered in docs/reflection.md. Each answer is **half a page maximum**.

1. Your pipeline crashes during processing, after writing to the raw zone but before writing to the curated zone. What is the impact on the data? Which checkpoint strategy prevents this issue?
2. You scale the producer up to 50,000 messages per second. In your opinion, what would be the first bottlenecks in your current architecture, and how would you fix them?
3. Compare the advantages and drawbacks of using Kafka as the *source of truth* for historical data, versus a Parquet data lake. In which scenarios should each be preferred?
4. A sensor breaks and emits aberrant values for 2 hours. How does your architecture detect this case? How would you isolate these data points without deleting them?
5. You must add a new sensor type `co2`. Which parts of your pipeline must be modified? Give a precise list of files and changes.

Appendix A — Useful commands

```
1 # Start the cluster
2 docker compose up -d
3
4 # List topics
5 docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 --list
6
7 # Describe the topic
8 docker exec kafka1 kafka-topics --bootstrap-server kafka1:29092 \
9     --describe --topic sensor-events
10
11 # Monitor a consumer group lag
12 docker exec kafka1 kafka-consumer-groups \
13     --bootstrap-server kafka1:29092 \
14     --describe --group <group-id>
15
16 # Submit the Spark job
17 spark-submit \
18     --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 \
19     src/spark_pipeline.py
20
21 # Test the API
22 curl -s http://localhost:5000/api/v1/health | python3 -m json.tool
```

Listing 4: Startup and inspection

Appendix B — Submission checklist

Before submitting, verify:

`docker compose up -d` starts all containers without errors

The `sensor-events` topic exists with 3 partitions and RF=3

`python src/producer.py -count 200` sends messages

`spark-submit src/spark_pipeline.py` starts the pipeline

The three data lake zones contain files

The API responds on `http://localhost:5000/api/v1/health`

All 6 endpoints are functional

`README.md` contains all the required sections

`requirements.txt` is up to date

No absolute user paths in the code

The zip archive is named `LASTNAME_FIRSTNAME_exam.zip`

Good luck and happy pipelining!

Remember to commit regularly.